

MODULARIZAÇÃO E ARQUIVOS – MINI CRM

Pensamento Computacional e Automação com Python



Oportunidades ?

Funil de vendas
Funil Padrão

Visão de trabalho
Funil de vendas

Responsável
Minhas oportunidades

Status da oportunidade
Em andamento

Sem contato ↻

3 oportunidades R\$ 0,00



Consultoria Tabajara
Consultoria Tabajara

R\$ 0,00

R\$ 0,00

★★★★★

☹

i



Escritório Topzera
Escritório Topzera

R\$ 0,00

R\$ 0,00

★★★★☆

☹

i



Escritório Supimpa
Escritório Supimpa

R\$ 0,00

R\$ 0,00

★★★★☆

☹

i

Contato feito ↻

1 oportunidades R\$ 10.000,00



Negócio Empresa Legal
Empresa Legal

R\$ 10.0...

R\$ 0,00

★★★★☆

☹

i

Identificação do interesse ↻

1 oportunidades R\$ 10.000,00



Empresa Massa
Empresa Massa

R\$ 10.0...

R\$ 0,00

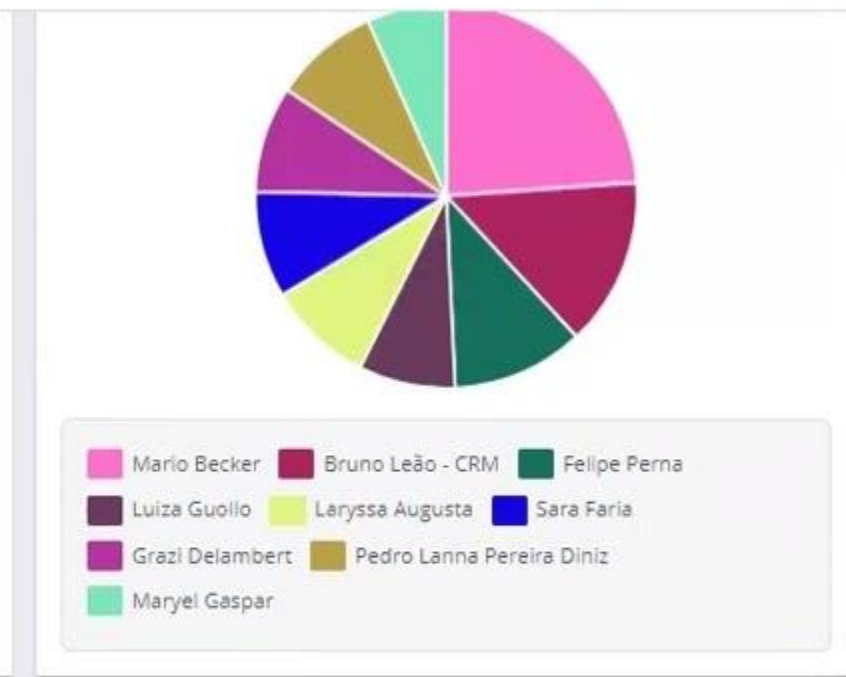
★★★★★

☹

i

Apresentação ↻

0 oportunidades R\$ 0,00



Construindo um Mini CRM com Python

De scripts isolados para um sistema real e modular.

```
def load_data(filepath):  
    with open(FILE, 'r') as f:  
        data = json.load(f)
```

```
class Client(BaseModel):  
    id: int  
    name: str  
    email: EmailStr
```

```
crm_system = CRM()  
crm.add_client(new_client)
```

```
crm.add_client(new_client)  
if not data:  
    raise DataError("Empty")
```



```
{  
    "id": 102,  
    "name": "Globex Inc",  
    "contact": "bob@globex.com",  
    "status": "pending"  
}
```

```
class Client(BaseModel):  
    id: int  
    name = EmailStr
```

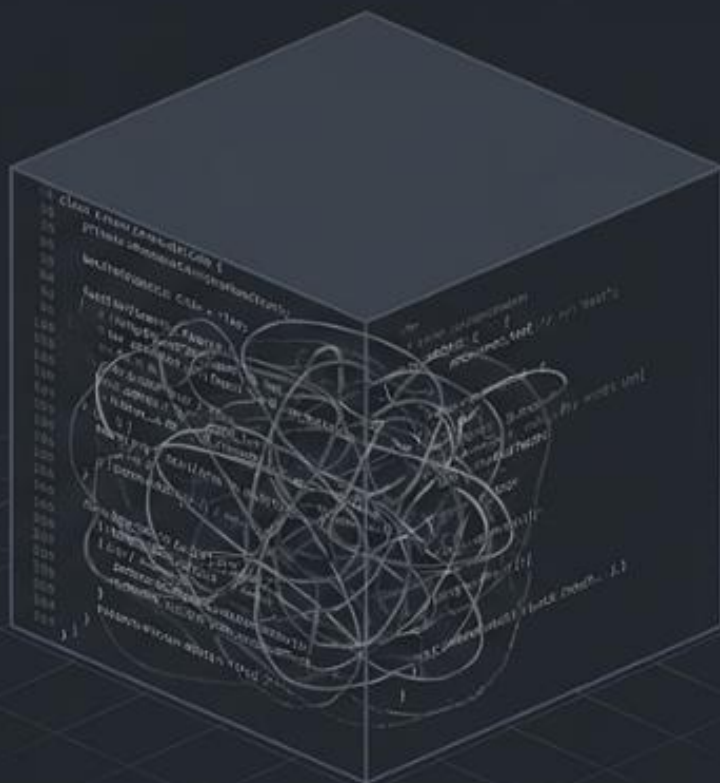
```
if not data:  
    raise DataError("Empty")
```

```
class Client(BaseModel):  
    id: int  
    name: str  
    email: EmailStr
```

```
def save_data(self):  
    json.dump(self.data, f, indent=4)
```


O fim do arquivo que faz tudo

Código Monolítico



Tudo em um único arquivo. Rápido para começar, impossível de manter ou evoluir.

Código Organizado

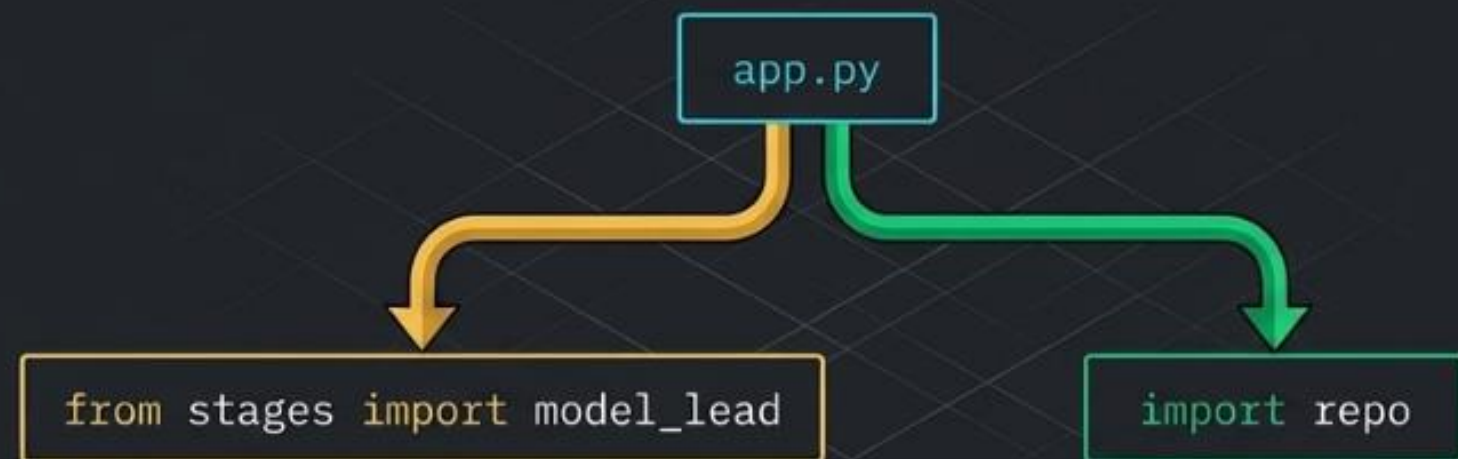


Cada arquivo tem uma responsabilidade única. O código se torna previsível, testável e profissional.

A anatomia do nosso sistema



Conectando as estações de trabalho



Não importamos apenas bibliotecas externas (como `random`).

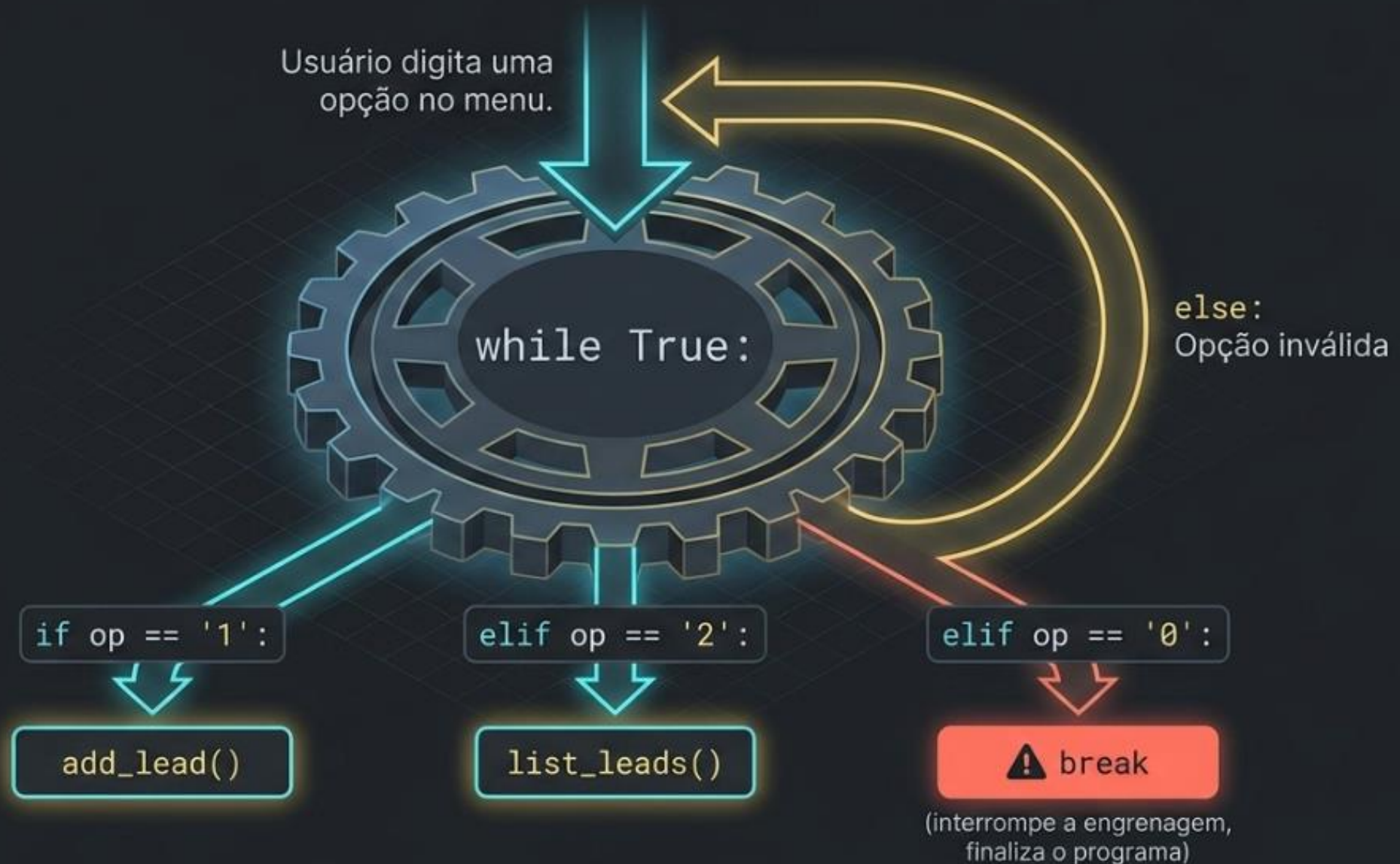
Importamos nosso próprio código.



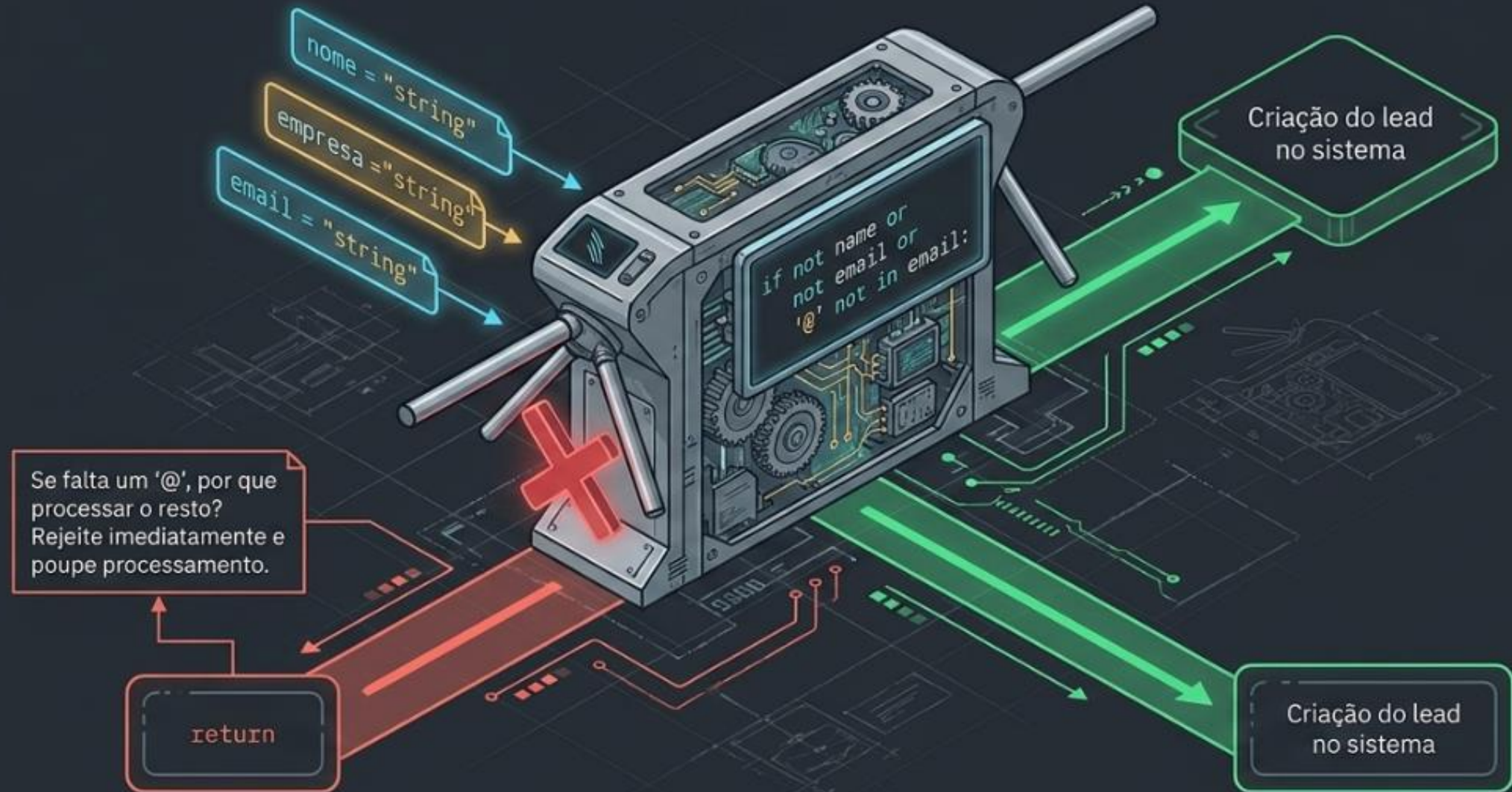
Evita que o menu ligue automaticamente se `app.py` for importado por outro sistema.

Só roda o `main()` se executado diretamente.

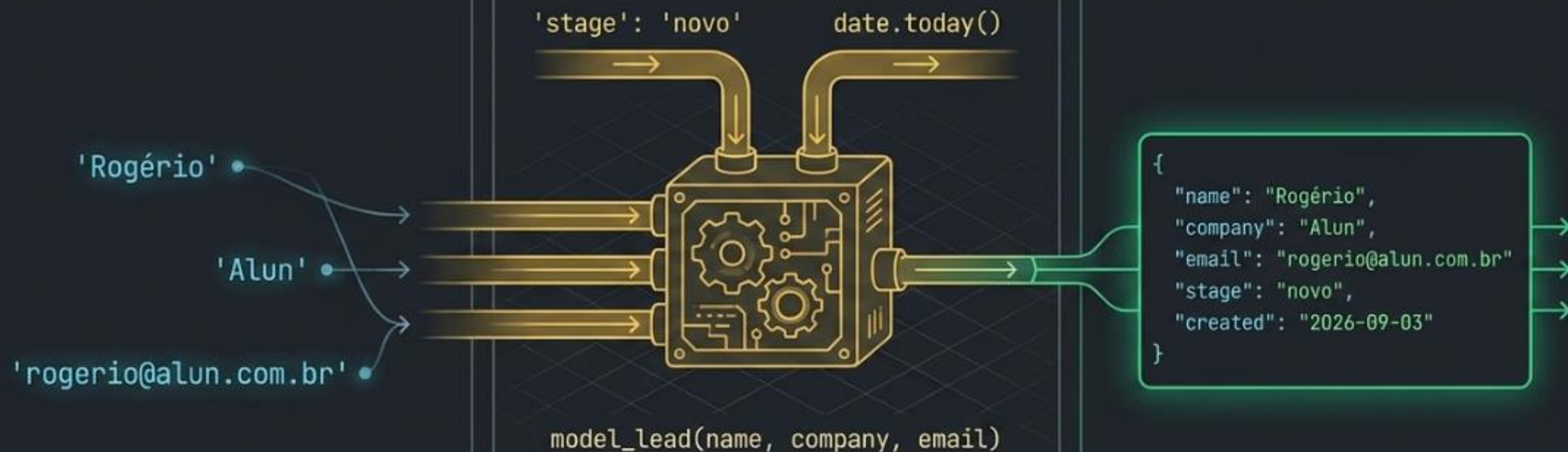
O loop infinito de interação



A catraca de validação: Fail Fast



Modelando o mundo com dicionários



O abismo da memória vs. a solidez do disco

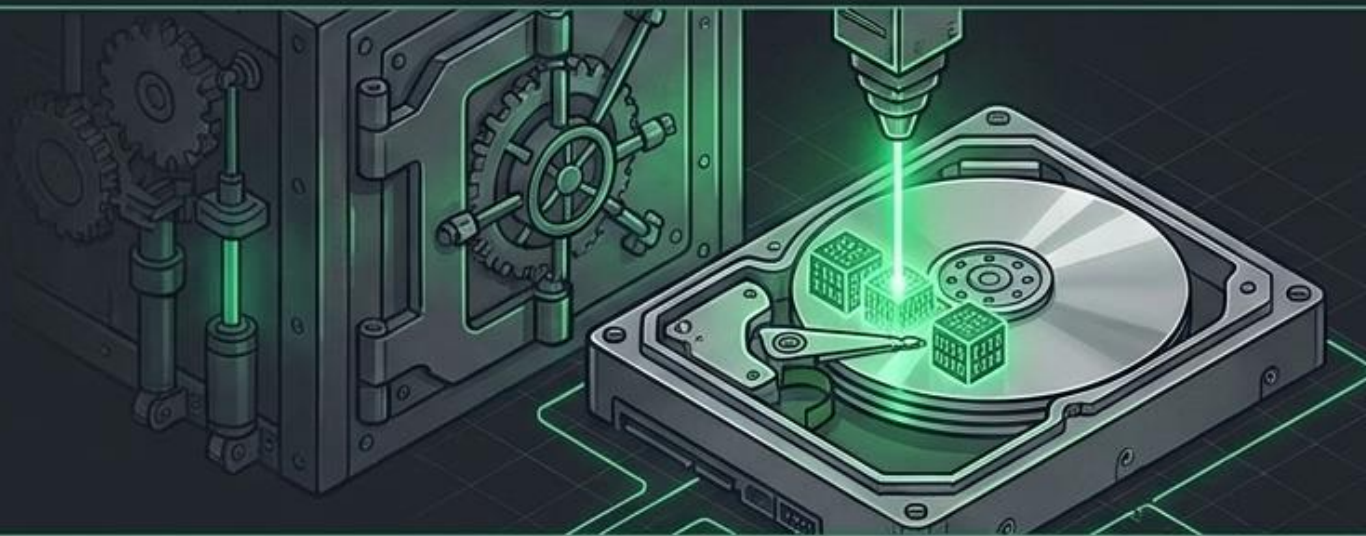
A Memória RAM

Na memória, seus `leads` sobrevivem apenas enquanto o programa estiver aberto.



A Persistência

A persistência de dados (salvar em arquivo/banco) é o que faz a informação `existir` amanhã.

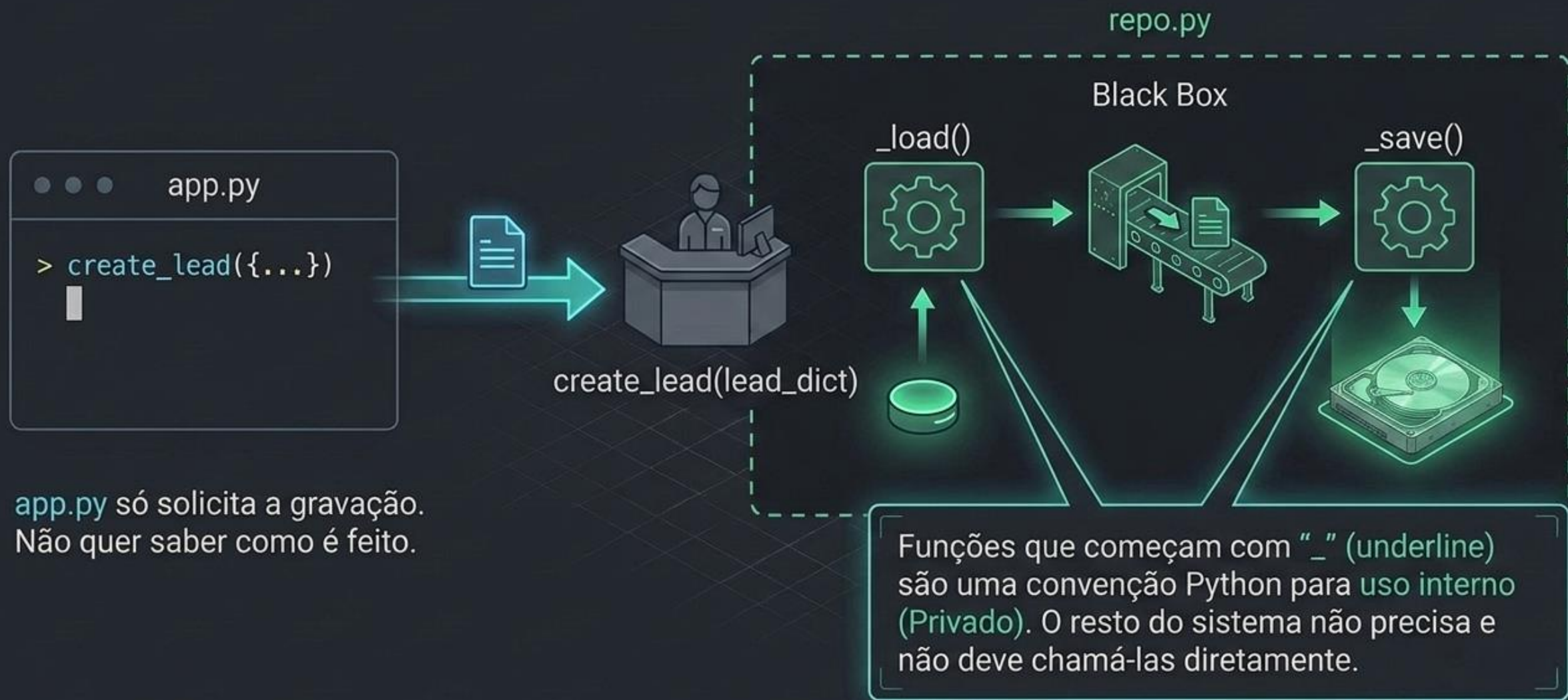


JSON: A linguagem universal dos sistemas



O JSON (JavaScript Object Notation) transcende o Python. É a mesma estrutura que você usará no futuro para conversar com qualquer API na web.

O Repositório: Escondendo a complexidade



Navegação à prova de falhas com Pathlib

```
DATA_DIR = Path(__file__).resolve().parent / 'data'
```



Independentemente de onde o usuário rodar o terminal (Desktop, Downloads, etc), o sistema encontra os arquivos corretamente.

Tratando o caos: Interceptação de Erros



O sistema sobrevive e recomeça graciosamente com uma lista vazia, uma decisão de design consciente para proteger a aplicação.

Listando e formatando dados com elegância

```
for i, l in enumerate(leads):
```

`i`

`l`

```
{'name': '...',  
'company': '...'}
```

```
f'{i:02d} | {l['name']:<20} | {l['company']:<17}'
```

```
00 | Rogério           | Alun           | rogerio@alun.com.br
```

Visão Panorâmica: O Ciclo de Vida do Dado



O que realmente construímos hoje?



A sintaxe pode mudar e as ferramentas vão escalar.
Mas a arquitetura mental (separar responsabilidades, modularizar, persistir) será a mesma.
Estruturar o hoje é o segredo para conseguir evoluir amanhã.

ALGUMA DÚVIDA?

Entre em contato pelo Teams ou por e-mail



<https://github.com/alexandrerussi>



Alexandre Russi Junior

Projects Coordinator | Higher Education Professor



[/alexandrerussi](#)